

FPC 3.2.2 Referenz Handbuch

Inhaltsverzeichnis

Lizenz - Bitte lesen !!!	3
Überblick	5
Lizenzen - 'LGPL'	6
FAQ - Questions-Answers	9
1 - Basic Questions	12
Lizenzen - MIT	12
Deutsch	13
Englisch	14
Französisch	15
Spanisch	16
Polnisch	17
Russisch	18
Inhalt	19
Liste der Tabellen	24
Über dieses Handbuch	24
Bezeichnungen	24
Syntax Diagramme	24
Über die Sprache Pascal	25
1. Pascal Zeichen und Symbole	26
2. Konstanten	26
3. Typen	26
4. Variablen	26
5. Objekte	26
6. Klassen	26
7. Schnittstellen (Interface's)	27
8. Generics	27
9. Erweiterte Record's	27
10. Klassen, Record's, und Typen-Helfer	27
11. Objektorientierte Pascal - Klassen	27
12. Ausdrücke	27
13. Anweisungen	27
14. Benutzung von Funktionen und Prozeduren	27
15. Operatoren Überladung	27
16. Programme, Module und Blöcke	27
17. Ausnahmen	27
18. Assembler	27
18.1. Anweisungen	27
18.2. Prozeduren und Funktionen	27

Lizenz

Bitte lesen Sie diese Lizenz gründlich durch, bevor Sie Lazarus die Dokumentation oder Teile dieser Software verwenden wollen. Sollten Sie nicht mit den hier aufgeführten Bedingungen einverstanden sein, ist die Nutzung oder Verwendung des Quellcodes zu diesen Produkt (oder auch Teile davon) nicht gestattet.

Bei der Verwendung darf kein kommerzieller Zweck der Gewinnerzielung entstehen !

Freeware und Shareware-Programme zeichnen sich vor allem dadurch aus, dass sie kostenlos beziehungsweise gegen einen relativ geringen Preis dem Nutzer die Verwendung teilweise hochwertiger Computerprogramme ermöglichen, die sich durchaus mit kommerziellen Produkten messen lassen können. Daher sind diese Softwaretypen gerade in Zeiten begrenzter finanzieller Mittel - auch im schulischen Umfeld sehr beliebt und kommen in vielfältigen Gebieten zum Einsatz (zum Beispiel bei der Netzwerkadministration, bei der Einrichtung von Servern, bei der Installation von Internet-Software auf den Nutzer-PCs oder bei Office-Anwendungen).

Da es sich auch bei Free- und Sharewareprogrammen um Computerprogramme handelt, die gemäß § 69a Urheberrechtsgesetz (UrhG) urheberrechtlich geschützt sind, liegt es bei diesen Softwareprodukten weitgehend in den Händen des Rechteinhabers zum Beispiel des Programmiers - zu bestimmen, in welchem Umfang diese durch dritte Personen genutzt werden dürfen.

Dabei können Freeware- und Sharewareprogramme nach den Lizenzbedingungen in der Regel beliebig kopiert und weitergegeben werden, dagegen ist vor allem eine Veränderung der Programme üblicherweise nicht gestattet. Es ist deshalb eigentlich auch nicht korrekt, wenn im Zusammenhang mit Free- und Shareware immer wieder der Begriff der "Public-Domain-Software" verwendet wird, der frei übersetzt "Software, die im öffentlichen Eigentum steht" bedeutet. Denn dies impliziert, dass die Software von der Öffentlichkeit beliebig genutzt und damit auch verändert werden darf (also gemeinfrei ist); letzteres ist bei Free- und Shareware aber gerade nicht der Fall.

Shareware kann man in zahlreiche Unterkategorien einteilen. So kann Shareware beispielsweise werbegestützte Software oder "Adware" sein, welche dem User Werbung anzeigen soll. Dies hat den Zweck Einnahmen für den Eigentümer zu generieren. Eine weitere Art bezeichnet man als Demoware, wobei es sich hier nur um eine Demoversion der Software handelt.

Dabei ist es oft eine Testversion mit einer festgelegten Zeitspanne ("Trialware"). In anderen Fällen kann die gesamte Funktionalität der Anwendung deaktiviert sein, so dass man zwar alle Funktionen sehen kannst, aber dafür bezahlen muss ("Crippleware").

Weiterhin gibt es auch "Freemium"-Shareware, bei welcher einige Funktionen in der kostenlosen Version verfügbar sind, Sie diese jedoch bezahlen müssen, um die volle Funktionalität freischalten zu können. Obwohl Shareware eine gute Option für jeden ist, der Software testen möchte, bevor man sich zu einem Kauf verpflichtet, sollten Sie dennoch vorsichtig sein. Denn Cyberkriminelle sind nur allzu bereit, den Eifer der Menschen auszunutzen, etwas gratis zu bekommen.

Dieses Versprechen freier Software ist eine gängige Social-Engineering-Taktik, mit der sie Internetnutzer dazu bringen, bösartige Software herunterzuladen.

Bei Shareware gibt es ebenso wie bei anderen Softwareprodukten keine Garantie dafür, dass die Software virenfrei ist. Aus diesem Grund ist es wichtig, dass der Benutzer sein Antivirenprogramm auf dem aktuellen Stand hält, bevor er Shareware herunterlädt.

Prinzipiell kann jeder Shareware herunterladen, solange er einen Computer hat, der die Systemanforderungen des Programms erfüllt. Es gibt jedoch einige Seiten, die nur Benutzern in bestimmten Ländern das Herunterladen von Shareware erlauben.

Nicht alle Shareware-Programme werden überall zum Kauf angeboten. Der beste Weg, um zu sehen, wo man ein bestimmtes Programm kaufen kann, ist, die Website des Entwicklers zu besuchen und herauszufinden, auf welchen Plattformen das Programm erhältlich ist.

Freeware ist Software, die kostenlos heruntergeladen werden kann, aber keine Lizenzgebühren mehr verlangt. Im Gegensatz dazu erfordert Shareware normalerweise eine Lizenzgebühr für den vollen Zugang zu allen Funktionen. Beide Modelle ermöglichen es Benutzern, Software kostenlos auszuprobieren, aber

Shareware ist normalerweise etwas umfangreicher und bietet mehr Funktionen.

Dieses Software-Produkt besitzt eine Hybrid-Lizenz zwischen Free- und Share-Ware. Sie können es kostenlos einsetzen und an andere Personen weitergeben, solange Sie keine Teile der Software ändern oder kopieren. Es bedarf einer schriftlichen Einwilligung des Entwicklers (Jens Kallup), sollten Teile geändert oder übernommen werden.

Sie können dieses Produkt auf mehreren lokalen Computer gleichzeitig nutzen. Eine öffentlich zugängliche Nutzung ist nicht bestrebt mit der aktuellen Version.

Obwohl bei der Entwicklung dieser Software viel Sorgfalt gepflegt wurde, können Fehler nicht ausgeschlossen werden.

Es werden daher keine Garantien auf entstanden Schäden oder Kosten übernommen, die während der Verwendung dieser Software entstehen. Alles erfolgt auf Eigenes Risiko !!!

Für zukünftige Entwicklungen können Sie jedoch einen kleinen Obolus in Form von Spenden hinterlegen.

Überblick

Free Pascal ist ein ausgereifter, vielseitiger, quelloffener Pascal-Compiler. Er kann für viele Prozessorarchitekturen eingesetzt werden: Intel x86 (16 und 32 bit), AMD64/x86-64, PowerPC, PowerPC64, SPARC, SPARC64, ARM, AArch64, MIPS, Motorola 68k, AVR, und die JVM. Zu den unterstützten Betriebssystemen gehören Windows (16/32/64 Bit, CE und natives NT), Linux, Mac OS X/iOS/iPhoneSimulator/Darwin, FreeBSD und andere BSD-Varianten, DOS (16 Bit oder 32 Bit DPMI), OS/2, AIX, Android, Haiku, Nintendo GBA/DS/Wii, AmigaOS, MorphOS, AROS, Atari TOS und verschiedene Embedded-Plattformen. Zusätzlich ist in der Entwicklungsversion Unterstützung für RISC-V (32/64), Xtensa und Z80 Architekturen und für die LLVM Compiler Infrastruktur verfügbar. Zusätzlich betreut das Free Pascal Team einen Transpiler für Pascal nach Javascript namens pas2js.

Aktuelle Version

Version 3.2.2 ist die letzte stabile Version von Free Pascal. Klicken Sie auf den **Download-Link** und wählen Sie einen Spiegel in Ihrer Nähe, um Ihre Kopie herunterzuladen. Die Entwicklungsversionen haben die Versionsnummern 3.3.x. Auf der Entwicklungsseite erfahren Sie, wie Sie die neuesten Quellen erhalten und die Entwicklung unterstützen können.

Funktionen

Die Sprachsyntax ist sehr gut kompatibel mit TP 7.0 sowie mit den meisten Versionen von Delphi (Klassen, rtti, Exceptions, AnsiString's, WideString's, Interfaces). Es gibt auch einen Mac Pascal Modus, der weitgehend kompatibel mit Think Pascal und MetroWerks Pascal ist. Darüber hinaus unterstützt Free Pascal Funktionsüberladung, Operatorüberladung, globale Eigenschaften und einige andere zusätzliche Funktionen.

Erforderlich

x86-Architektur:

Für die 80x86-Version ist mindestens ein 386er Prozessor erforderlich, ein 486er wird jedoch empfohlen.

Die Mac OS X-Version erfordert Mac OS X 10.4 oder höher, wobei die Entwicklerwerkzeuge installiert sein müssen.

PowerPC-Architektur:

Jeder PowerPC-Prozessor ist geeignet. 16 MB RAM sind erforderlich. Es wird erwartet, dass die Mac OS classic Version mit System 7.5.3 und höher funktioniert. Die Mac OS X Version benötigt Mac OS X 10.3 oder später (kann für 10.2.8 oder später kompiliert werden), mit den installierten Entwicklerwerkzeugen. Auf anderen Betriebssystemen läuft Free Pascal auf jedem System, das das Betriebssystem ausführen kann.

ARM-Architektur:

16 MB RAM sind erforderlich. Läuft auf jeder ARM-Linux-Installation.

Sparc-Architektur:

16 MB RAM sind erforderlich. Läuft auf jeder Sparc-Linux-Installation (Solaris ist experimentell).

Lizenz

Die Pakete und die Laufzeitbibliothek stehen unter einer modifizierten Library GNU Public License, um die Verwendung statischer Bibliotheken bei der Erstellung von Anwendungen zu ermöglichen. Der Quellcode des Compilers selbst steht unter der GNU General Public License. Die Quellen sowohl für den Compiler als auch für die Laufzeitbibliothek sind verfügbar; der komplette Compiler ist in Pascal geschrieben.

GNU Lesser PUBLIC LISENZ Version 3

Version 3

Version 2.1

Warum man die Lesser GPL nicht für die nächste Bibliothek verwenden sollte
Bitte beachten Sie für weitere allgemeine Empfehlungen über die Wahl einer Lizenz für Ihr Werk unseren [Lizenzratgeber](#).

Das GNU-Projekt verfügt über zwei wesentliche Lizenzen zur Verwendung mit Bibliotheken. Eine ist die GNU Lesser General Public License (LGPL), die andere ist die normale GNU General Public License (GPL). Die Wahl der Lizenz macht einen großen Unterschied: mithilfe der GNU LGPL wird die Nutzung der Bibliothek in proprietären Programmen erlaubt, mithilfe der GNU GPL ist sie ausschließlich für freie Programme nutzbar.

Welche Lizenz am besten für eine bestimmte Bibliothek geeignet ist, ist eine Frage der Strategie und situationsabhängig. Derzeit werden die meisten GNU-Bibliotheken von der GNU LGPL abgedeckt, und das bedeutet, es wird nur eine dieser zwei Strategien verwendet, die andere wird vernachlässigt. Deshalb suchen wir nun weitere Bibliotheken, um sie **unter der normalen GNU GPL** freizugeben.

Entwickler proprietärer Software haben den Vorteil des Geldes; Entwickler freier Software müssen Vorteile für einander schaffen. Das Verwenden der GNU GPL für eine Bibliothek gibt Freie-Software-Entwicklern gegenüber proprietären Entwicklern einen Vorteil: eine Bibliothek, die sie nutzen können, während proprietäre Entwickler sie nicht nutzen können.

Die Verwendung der normalen GNU GPL ist nicht für jede Bibliothek vorteilhaft. Es gibt Gründe, in denen es in bestimmten Fällen besser sein kann, die GNU LGPL zu verwenden. Häufigster Fall ist, wenn die Funktionen einer freien Bibliothek für proprietäre Software durch andere Bibliotheken leicht zugänglich sind. In diesem Fall kann die Bibliothek freier Software keinen besonderen Vorteil geben, so dass es besser ist für diese Bibliothek die GNU LGPL zu verwenden.

Deshalb haben wir die GNU LGPL für die GNU C-Bibliothek eingesetzt. Immerhin gibt es viele andere C-Bibliotheken. Mithilfe der GNU GPL wären Entwickler proprietärer Software gezwungen gewesen, eine andere zu verwenden - kein Problem für sie, nur für uns.

Wenn eine Bibliothek jedoch eine signifikante einzigartige Fähigkeit enthält, wie beispielsweise **GNU Readline**, ist das etwas ganz anderes. Die Readline-Bibliothek implementiert Eingabeergänzung und -verlauf für interaktive Programme, und das ist eine woanders nicht allgemein verfügbare Funktion. Die Freigabe unter GNU GPL - und damit die ausschließliche Nutzung in freien Programmen - gibt unserer Gemeinschaft einen richtigen Schub. Mindestens ein Anwendungsprogramm ist heute insbesondere freie Software, weil das für die Nutzung von Readline notwendig war.

Wenn wir eine Auswahl leistungsfähiger, unter GPL lizenzierten Bibliotheken ansammeln, die keine parallelen für proprietäre Software haben, liefern sie eine Reihe nützlicher Module, die als Bausteine in neuen freien Programmen dienen. Dies wird ein signifikanter Vorteil für die weitere Entwicklung freier Software sein, und einige Projekte werden sich dazu entscheiden, Software frei zu machen, um diese Bibliotheken zu nutzen. Universitäre Projekte können leicht beeinflusst werden. Heutzutage, wo Unternehmen beginnen daran zu denken Software frei zu machen, können sogar einige kommerzielle Projekte auf diese Weise beeinflusst werden.

Entwickler proprietärer Software - bestrebt, dem freien Wettbewerb einen wichtigen Vorteil zu versagen - werden versuchen Autoren davon zu überzeugen, keine GPL-lizenzierten Bibliotheken der Sammlung beizutragen. Beispielsweise könnten sie an das Ego appellieren, wenn sie *mehr Benutzer für diese Bibliothek* versprechen, wenn wir sie den Quellcode in proprietären Softwareprodukten verwenden lassen. Popularität ist verlockend und für einen Entwickler einer Bibliothek die Idee leicht zu rationalisieren, dass die Erhöhung der Popularität dieser einen Bibliothek zu steigern das ist, was die Gemeinschaft vor allem braucht.

Aber wir sollten nicht auf diese Versuchungen hören, weil wir viel mehr erreichen können, wenn wir zusammenstehen. Wir Freie-Software-Entwickler sollten einander unterstützen. Durch die Freigabe von

Bibliotheken, die nur auf freie Software begrenzt sind, helfen wir einander, dass die eigenen Freie-Software-Pakete die proprietären Pendanten übertreffen. Die gesamte Freie-Software-Bewegung wird mehr Popularität haben, weil sich freie Software als Ganzes besser gegen den Wettbewerb abheben wird.

GNU Lesser PUBLIC LISENZ Version 3

Version 3

Version 2.1

Warum man die Lesser GPL nicht für die nächste Bibliothek verwenden sollte
Bitte beachten Sie für weitere allgemeine Empfehlungen über die Wahl einer Lizenz für Ihr Werk unseren Lizenzratgeber.

Das GNU-Projekt verfügt über zwei wesentliche Lizenzen zur Verwendung mit Bibliotheken. Eine ist die GNU Lesser General Public License (LGPL), die andere ist die normale GNU General Public License (GPL). Die Wahl der Lizenz macht einen großen Unterschied: mithilfe der GNU LGPL wird die Nutzung der Bibliothek in proprietären Programmen erlaubt, mithilfe der GNU GPL ist sie ausschließlich für freie Programme nutzbar.

Welche Lizenz am besten für eine bestimmte Bibliothek geeignet ist, ist eine Frage der Strategie und situationsabhängig. Derzeit werden die meisten GNU-Bibliotheken von der GNU LGPL abgedeckt, und das bedeutet, es wird nur eine dieser zwei Strategien verwendet, die andere wird vernachlässigt. Deshalb suchen wir nun weitere Bibliotheken, um sie **unter der normalen GNU GPL** freizugeben.

Entwickler proprietärer Software haben den Vorteil des Geldes; Entwickler freier Software müssen Vorteile für einander schaffen. Das Verwenden der GNU GPL für eine Bibliothek gibt Freie-Software-Entwicklern gegenüber proprietären Entwicklern einen Vorteil: eine Bibliothek, die sie nutzen können, während proprietäre Entwickler sie nicht nutzen können.

Die Verwendung der normalen GNU GPL ist nicht für jede Bibliothek vorteilhaft. Es gibt Gründe, in denen es in bestimmten Fällen besser sein kann, die GNU LGPL zu verwenden. Häufigster Fall ist, wenn die Funktionen einer freien Bibliothek für proprietäre Software durch andere Bibliotheken leicht zugänglich sind. In diesem Fall kann die Bibliothek freier Software keinen besonderen Vorteil geben, so dass es besser ist für diese Bibliothek die GNU LGPL zu verwenden.

Deshalb haben wir die GNU LGPL für die GNU C-Bibliothek eingesetzt. Immerhin gibt es viele andere C-Bibliotheken. Mithilfe der GNU GPL wären Entwickler proprietärer Software gezwungen gewesen, eine andere zu verwenden - kein Problem für sie, nur für uns.

Wenn eine Bibliothek jedoch eine signifikante einzigartige Fähigkeit enthält, wie beispielsweise **GNU Readline**, ist das etwas ganz anderes. Die Readline-Bibliothek implementiert Eingabeergänzung und -verlauf für interaktive Programme, und das ist eine woanders nicht allgemein verfügbare Funktion. Die Freigabe unter GNU GPL - und damit die ausschließliche Nutzung in freien Programmen - gibt unserer Gemeinschaft einen richtigen Schub. Mindestens ein Anwendungsprogramm ist heute insbesondere freie Software, weil das für die Nutzung von Readline notwendig war.

Wenn wir eine Auswahl leistungsfähiger, unter GPL lizenzierten Bibliotheken ansammeln, die keine parallelen für proprietäre Software haben, liefern sie eine Reihe nützlicher Module, die als Bausteine in neuen freien Programmen dienen. Dies wird ein signifikanter Vorteil für die weitere Entwicklung freier Software sein, und einige Projekte werden sich dazu entscheiden, Software frei zu machen, um diese Bibliotheken zu nutzen. Universitäre Projekte können leicht beeinflusst werden. Heutzutage, wo Unternehmen beginnen daran zu denken Software frei zu machen, können sogar einige kommerzielle Projekte auf diese Weise beeinflusst werden.

Entwickler proprietärer Software - bestrebt, dem freien Wettbewerb einen wichtigen Vorteil zu versagen - werden versuchen Autoren davon zu überzeugen, keine GPL-lizenzierten Bibliotheken der Sammlung beizutragen. Beispielsweise könnten sie an das Ego appellieren, wenn sie *mehr Benutzer für diese Bibliothek* versprechen, wenn wir sie den Quellcode in proprietären Softwareprodukten verwenden lassen. Popularität ist verlockend und für einen Entwickler einer Bibliothek die Idee leicht zu rationalisieren, dass die Erhöhung der Popularität dieser einen Bibliothek zu steigern das ist, was die Gemeinschaft vor

allem braucht.

Aber wir sollten nicht auf diese Versuchungen hören, weil wir viel mehr erreichen können, wenn wir zusammenstehen. Wir Freie-Software-Entwickler sollten einander unterstützen. Durch die Freigabe von Bibliotheken, die nur auf freie Software begrenzt sind, helfen wir einander, dass die eigenen Freie-Software-Pakete die proprietären Pendants übertreffen. Die gesamte Freie-Software-Bewegung wird mehr Popularität haben, weil sich freie Software als Ganzes besser gegen den Wettbewerb abheben wird.

FAQ - Question and Answer

1.1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

2.1. General understanding of the GNU licenses

3.1. Using GNU licenses for your programs

4.1. Distribution of programs released under the GNU licenses

5.1. Using programs released under the GNU licenses when writing other programs

6.1. Combining work with code released under the GNU licenses

7.1. Questions about violations of the GNU licenses

1.1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

1.1.1. What does GPL stand for?

1.1.2. Does free software mean using the GPL?

1.1.3. Why should I use the GNU GPL rather than other free software licenses?

1.1.4. Does all GNU software use the GNU GPL as its license?

1.1.5. Does using the GPL for a program make it GNU software?

1.1.6. Can I use the GPL for something other than software?

1.1.7. Why don't you use the GPL for manuals?

1.1.8. Are there translations of the GPL into other languages?

1.1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?

1.1.10. Who has the power to enforce the GPL?

1.1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPLed program, should I do this, too? If so, how?

1.1.12. Can I modify the GPL and make a modified license?

1.1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

2.1. General understanding of the GNU licenses

2.1.1. Why does the GPL permit users to publish their modified versions?

2.1.2. Does the GPL require that source code of modified versions be posted to the public?

2.1.3. Can I have a GPL-covered program and an unrelated nonfree program on the same computer?

2.1.4. If I know someone has a copy of a GPL-covered program, can I demand they give me a copy?

2.1.5. What does "written offer valid for any third party" mean in GPLv2? Does that mean everyone in the world can get

	<i>the source to any GPLed program no matter what?</i>
2.1.6.	<i>The GPL says that modified versions, if released, must be "licensed to all third parties." Who are these third parties?</i>
2.1.7.	<i>Does the GPL allow me to sell copies of the program for money?</i>
2.1.8.	<i>Does the GPL allow me to charge a fee for downloading the program from my distribution site?</i>
2.1.9.	<i>Does the GPL allow me to require that anyone who receives the software must pay me a fee and/or notify me?</i>
2.1.10.	<i>If I distribute GPLed software for a fee, am I required to also make it available to the public without a charge?</i>
2.1.11.	<i>Does the GPL allow me to distribute a copy under a nondisclosure agreement?</i>
2.1.12.	<i>Does the GPL allow me to distribute a modified or beta version under a nondisclosure agreement?</i>
2.1.13.	<i>Does the GPL allow me to develop a modified version under a nondisclosure agreement?</i>
2.1.14.	<i>Why does the GPL require including a copy of the GPL with every copy of the program?</i>
2.1.15.	<i>What if the work is not very long?</i>
2.1.16.	<i>Am I required to claim a copyright on my modifications to a GPL-covered program?</i>
2.1.17.	<i>What does the GPL say about translating some code to a different programming language?</i>
2.1.18.	<i>If a program combines public-domain code with GPL-covered code, can I take the public-domain part and use it as public domain code?</i>
2.1.19.	<i>I want to get credit for my work. I want people to know what I wrote. Can I still get credit if I use the GPL?</i>
2.1.20.	<i>Does the GPL allow me to add terms that would require citation or acknowledgment in research papers which use the GPL-covered software or its output?</i>
2.1.21.	<i>Can I omit the preamble of the GPL, or the instructions for how to use it on your own programs, to save space?</i>
2.1.22.	<i>What does it mean to say that two licenses are "compatible"?</i>
2.1.23.	<i>What does it mean to say a license is "compatible with the GPL"?</i>
2.1.24.	<i>Why is the original BSD license incompatible with the GPL?</i>
2.1.25.	<i>What is the difference between an "aggregate" and other kinds of "modified versions"?</i>
2.1.26.	<i>When it comes to determining whether two pieces of software form a single work, does the fact that the code is in one or more containers have any effect?</i>
2.1.27.	<i>Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPLed program, should I do this, too? If so, how?</i>
2.1.28.	<i>If I use a piece of software that has been obtained under the GNU GPL, am I allowed to modify the original code into a new program, then distribute and sell that new program commercially?</i>
2.1.29.	<i>Can I use the GPL for something other than software?</i>

2.1.30. I'd like to license my code under the GPL, but I'd also like to make it clear that it can't be used for military and/or commercial uses. Can I do this?

2.1.31. Can I use the GPL to license hardware?

2.1.32. Does prelinking a GPLed binary to various libraries on the system, to optimize its performance, count as modification?

2.1.33. How does the LGPL work with Java?

2.1.34. Why did you invent the new terms "propagate" and "convey" in GPLv3?

2.1.35. Is "convey" in GPLv3 the same thing as what GPLv2 means by "distribute"?

2.1.36. If I only make copies of a GPL-covered program and run them, without distributing or conveying them to others, what does the license require of me?

2.1.37. GPLv3 gives "making available to the public" as an example of propagation. What does this mean? Is making available a form of conveying?

2.1.38. Since distribution and making available to the public are forms of propagation that are also conveying in GPLv3, what are some examples of propagation that do not constitute conveying?

2.1.39. How does GPLv3 make BitTorrent distribution easier?

2.1.40. What is tivoization? How does GPLv3 prevent it?

2.1.41. Does GPLv3 prohibit DRM?

2.1.42. Does GPLv3 require that voters be able to modify the software running in a voting machine?

2.1.43. Does GPLv3 have a "patent retaliation clause"?

2.1.44. In GPLv3 and AGPLv3, what does it mean when it says "notwithstanding any other provision of this License"?

2.1.46. In AGPLv3, what counts as "interacting with [the software] remotely through a computer network"?

2.1.47. How does GPLv3's concept of "you" compare to the definition of "Legal Entity" in the Apache License 2.0?

2.1.48. In GPLv3, what does "the Program" refer to? Is it every program ever released under GPLv3?

2.1.49. If some network client software is released under AGPLv3, does it have to be able to provide source to the servers it interacts with?

2.1.50. or software that runs a proxy server licensed under the AGPL, how can I provide an offer of source to users interacting with that code?

3.1. Using GNU licenses for your programs

4.1. Distribution of programs released under the GNU licenses

5.1. Using programs released under the GNU licenses when writing other programs

6.1. Combining work with code released under the GNU licenses

7.1. Questions about violations of the GNU licenses

1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

- 1.1. What does GPL stand for?
- 1.2. Does free software mean using the GPL?
- 1.3. Why should I use the GNU GPL rather than other free software licenses?
- 1.4. Does all GNU software use the GNU GPL as its license?
- 1.5. Does using the GPL for a program make it GNU software?
- 1.6. Can I use the GPL for something other than software?
- 1.7. Why don't you use the GPL for manuals?
- 1.8. Are there translations of the GPL into other languages?
- 1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?
- 1.10. Who has the power to enforce the GPL?
- 1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPlEd program, should I do this, too? If so, how?
- 1.12. Can I modify the GPL and make a modified license?
- 1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

1.1. What does GPL stand for?

SSSS

SSSS

1.2. Does free software mean using the GPL?

1.3. Why should I use the GNU GPL rather than other free software licenses?

1.4. Does all GNU software use the GNU GPL as its license?

1.5. Does using the GPL for a program make it GNU software?

1.6. Can I use the GPL for something other than software?

1.7. Why don't you use the GPL for manuals?

1.8. Are there translations of the GPL into other languages?

1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?

1.10. Who has the power to enforce the GPL?

1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPlEd program, should I do this, too? If so, how?

1.12. Can I modify the GPL and make a modified license?

1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

MIT-Lizenzen / MIT Licenses

Copyright (c) 2025 Jens Kallup

**Deutsch
Englisch
Französisch
Spanisch
Polnisch
Russisch**

MIT-Lizenz / MIT License

Deutsche Übersetzung (inoffiziell)

Copyright (c) 2025 Jens Kallup

Hiermit wird jeder Person, die eine Kopie dieser Software und der zugehörigen Dokumentationsdateien (die "Software") erhält, kostenlos die Erlaubnis erteilt, uneingeschränkt mit der Software zu handeln, einschließlich und ohne Einschränkung des Rechts, sie zu verwenden, zu kopieren, zu modifizieren, zusammenzuführen, zu veröffentlichen, zu verbreiten, zu unterlizenzieren und/oder zu verkaufen, und Personen, denen diese Software zur Verfügung gestellt wird, dies zu erlauben, unter den folgenden Bedingungen:

Der obige Urheberrechtshinweis und dieser Genehmigungshinweis sind in allen Kopien oder wesentlichen Teilen der Software beizufügen.

DIE SOFTWARE WIRD OHNE JEDE AUSDRÜCKLICHE ODER IMPLIZIERTE GARANTIE BEREITGESTELLT, EINSCHLIESSLICH DER GARANTIE DER MARKTGÄNGIGKEIT, DER EIGNUNG FÜR EINEN BESTIMMTEN ZWECK UND DER NICHTVERLETZUNG. IN KEINEM FALL SIND DIE AUTOREN ODER COPYRIGHTINHABER FÜR JEDLICHEN SCHADEN ODER SONSTIGE ANSPRÜCHE HAFTBAR, SEI ES AUFGRUND EINER VERTRAGSKLAGE, EINER UNERLAUBTEN HANDLUNG ODER ANDERWEITIG, DER AUS DER SOFTWARE ODER DER VERWENDUNG ODER ANDEREN GESCHÄFTEN MIT DER SOFTWARE ENTSTEHT.

Original English Version

Copyright (c) 2025 Jens Kallup

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Traduction française (traduction non officielle)

Copyright (c) 2025 Jens Kallup

La permission est accordée, gratuitement, à toute personne obtenant une copie de ce logiciel et des fichiers de documentation associés (le "Logiciel"), de traiter dans le Logiciel sans restriction, y compris sans limitation les droits d'utiliser, de copier, de modifier, de fusionner, de publier, de distribuer, de sous-licencier et/ou de vendre des copies du Logiciel, et de permettre aux personnes à qui le Logiciel est fourni de le faire, sous réserve des conditions suivantes:

La mention de copyright ci-dessus et la présente autorisation doivent être incluses dans toutes les copies ou parties substantielles du Logiciel.

LE LOGICIEL EST FOURNI "EN L'ÉTAT", SANS GARANTIE D'AUCUNE SORTE, EXPRESSE OU IMPLICITE, Y COMPRIS MAIS SANS S'Y LIMITER AUX GARANTIES DE QUALITÉ MARCHANDE, D'ADÉQUATION À UN USAGE PARTICULIER ET D'ABSENCE DE CONTREFAÇON. EN AUCUN CAS, LES AUTEURS OU LES DÉTENTEURS DES DROITS D'AUTEUR NE PEUVENT ÊTRE TENUS RESPONSABLES DE TOUTE RÉCLAMATION, DOMMAGE OU AUTRE RESPONSABILITÉ, QUE CE SOIT DANS LE CADRE D'UN CONTRAT, D'UN DÉLIT OU AUTREMENT, DÉCOULANT DE, EN RELATION AVEC LE LOGICIEL OU L'UTILISATION OU AUTRES RELATIONS DANS LE LOGICIEL.

Traducción al español (traducción no oficial)

Copyright (c) 2025 Jens Kallup

Por la presente se concede permiso, libre de cargos, a cualquier persona que obtenga una copia de este software y los archivos de documentación asociados (el "Software"), para tratar con el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y para permitir a las personas a las que se les proporcione el Software hacerlo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso deberán incluirse en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A LAS GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O TITULARES DE LOS DERECHOS DE AUTOR SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRA RESPONSABILIDAD, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O DE OTRO TIPO, QUE SURJA DE, FUERA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTROS TRATOS EN EL SOFTWARE.

Tłumaczenie na język polski (nieoficjalne tłumaczenie)

Copyright (c) 2025 Jens Kallup

Niniejszym udziela się każdej osobie, która otrzyma kopię tego oprogramowania i powiązanych plików dokumentacji (dalej "Oprogramowanie"), bezpłatnego pozwolenia na korzystanie z Oprogramowania bez ograniczeń, w tym bez ograniczeń do praw używania, kopiowania, modyfikowania, łączenia, publikowania, dystrybuowania, sublicencjonowania i/lub sprzedaży kopii Oprogramowania, a także zezwalania osobom, którym Oprogramowanie jest udostępniane, na to samo, z zastrzeżeniem następujących warunków:

Powyższa informacja o prawach autorskich oraz niniejsza informacja o pozwoleniu muszą być dołączone do wszystkich kopii lub istotnych części Oprogramowania.

**OPROGRAMOWANIE JEST DOSTARCZANE „TAK JAK JEST”, BEZ JAKIEJKOLWIEK
GWARANCJI, WYRAŻNEJ
LUB DOROZUMIANEJ, W TYM MIĘDZY INNYMI GWARANCJI PRZYDATNOŚCI HANDLOWEJ,
PRZYDATNOŚCI
DO OKREŚLONEGO CELU ORAZ NARUSZENIA PRAW. W ŻADNYM WYPADKU AUTORZY LUB
POSIADACZE PRAW
AUTORSKICH NIE BĘDĄ ODPOWIEDZIALNI ZA JAKIEJKOLWIEK ROSZCZENIA, SZKODY LUB
INNE
ODPOWIEDZIALNOŚCI, CZY TO W RAMACH UMOWY, CZYNNYCH CZY INNYCH, WYNIKAJĄCYCH
Z LUB
ZWIĄZANYCH Z OPROGRAMOWANIEM LUB JEGO UŻYTKOWANIEM.**

Русский перевод (неофициальный перевод)

Copyright (c) 2025 Jens Kallup

Настоящим бесплатно предоставляется разрешение любому лицу, получившему копию данного программного обеспечения и сопутствующей документации (далее "Программное обеспечение"), использовать Программное обеспечение без ограничений, включая без ограничения права на использование, копирование, изменение, слияние, публикацию, распространение, sublicenzирование и/или продажу копий Программного обеспечения, а также предоставление таких прав лицам, которым предоставляется Программное обеспечение, при соблюдении следующих условий:

Вышеуказанное уведомление об авторском праве и данное уведомление о разрешении должны быть включены во все копии или значительные части Программного обеспечения.

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ "КАК ЕСТЬ", БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, ЯВНЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ, НО НЕ ОГРАНИЧИВАЯСЬ ГАРАНТИЯМИ ТОВАРНОЙ ПРИГОДНОСТИ, ПРИГОДНОСТИ ДЛЯ ОПРЕДЕЛЕННОЙ ЦЕЛИ И НЕНАРУШЕНИЯ ПРАВ. НИ ПРИ КАКИХ ОБСТОЯТЕЛЬСТВАХ АВТОРЫ ИЛИ ПРАВООБЛАДАТЕЛИ НЕ НЕСУТ ОТВЕТСТВЕННОСТИ ЗА КАКИЕ-ЛИБО ПРЕТЕНЗИИ, УБЫТКИ ИЛИ ИНЫЕ ОБЯЗАТЕЛЬСТВА, ВОЗНИКШИЕ В РЕЗУЛЬТАТЕ ДОГОВОРА, ДЕЛИКТА ИЛИ ИНЫМ ОБРАЗОМ, ВОЗНИКШИЕ ИЗ, ИЗ-ЗА ИЛИ В СВЯЗИ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ ИЛИ ИСПОЛЬЗОВАНИЕМ ИЛИ ИНЫМИ СДЕЛКАМИ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ.

Free Pascal Compiler v.3.2.2

Referenz Handbuch
Dokument v.3.2.2
März 2025

Übersetzung von Jens Kallup

Liste der Tabellen
Über diesen Leitfaden
Bezeichnungen
Syntax Diagramme
Über die Sprache Pascal

1. Pascal Zeichen und Symbole

- 1.1. Symbole
- 1.2. Kommentare
- 1.3. Reservierte Schlüsselwörter
 - 1.3.1. Turbo Pascal
 - 1.3.2. Object Pascal
 - 1.3.3. Modifikationen
- 1.4. Kennzeichnungen
- 1.5. Hinweise für Direktiven
- 1.6. Zahlen
- 1.7. Bezeichner
- 1.8. Zeichenketten

2. Konstanten

- 2.1. Gewöhnliche Konstanten
- 2.2. Typisierte
- 2.3. Ressourcen Zeichenketten

3. Typen

- 3.1. Basis-Typen
 - 3.1.1. Ordinale Typen
 - 3.1.2. Ganze Zahlen (Integer)
 - 3.1.3. Boolesche Typen
 - 3.1.4. Aufzählungen
 - 3.1.5. Untermengen
 - 3.1.6. Zeichen
- 3.2. Zeichen-Typen
 - 3.2.1. Char oder AnsiChar
 - 3.2.2. WideChar
 - 3.2.3. Sonstige
 - 3.2.4. Einzel-Byte Zeichenketten
 - 3.2.4.1. ShortString
 - 3.2.4.2. AnsiString
 - 3.2.4.3. Zeichen-Code Umwandlung
 - 3.2.4.4. RawByteString
 - 3.2.4.5. UTF8String
 - 3.2.5. Multi-Byte Zeichenketten
 - 3.2.5.1. UnicodeString
 - 3.2.5.2. WideString

- 3.2.6. Konstante Zeichenketten
- 3.2.7. Nullterminierende Zeichenketten (PChar)
- 3.2.8. Zeichenketten-Größen
- 3.3. Strukturierte Typen
 - 3.3.1. Gepackte Struktur-Typen
 - 3.3.2. Array's
 - 3.3.2.1. Statische Array's
 - 3.3.2.2. Dynamische Array's
 - 3.3.2.3. Typen-Kompatibilität dynamischer Array's
 - 3.3.2.4. Constuctor dynamischer Array's
 - 3.3.2.5. Feldkonstanten-Ausdrücke dynamischer Array's
 - 3.3.2.6. Packen und Entpacken eines Array's
 - 3.3.3. Record's
 - 3.3.3.1. Layout und Größe
 - 3.3.3.2. Bemerkungen und Beispiele
 - 3.3.4. Mengen-Typen
 - 3.3.6. Datei-Typen
- 3.4. Zeiger
- 3.5. Forward-Deklarationen
- 3.6. Prozedur-Typen
- 3.7. Variant's
 - 3.7.1. Definition
 - 3.7.1. Variant's in Zuweisungen und Ausdrücken
 - 3.7.3. Variant's im Interface-Teil
- 3.8. Alias-Typen
- 3.9. Verwaltete Typen
- 4. Variablen
 - 4.1. Definition
 - 4.2. Erklärung
 - 4.3. Geltungsbereich
 - 4.4. Initialisierte Variablen
 - 4.5. Initialisierte Variablen mit Standard-Wert
 - 4.6. Thread-Variablen
 - 4.7. Eigenschaften
- 5. Objekte
 - 5.1. Deklaration
 - 5.2. Abstrakte und Sealed Objekte
 - 5.3. Felder
 - 5.4. Klassen oder statische Felder
 - 5.5. Constructor und Destructor
 - 5.6. Methoden
 - 5.6.1. Erklärung
 - 5.6.2. Methoden-Aufruf
 - 5.6.2.1. Statische Methoden
 - 5.6.2.2. Virtuelle Methoden
 - 5.6.2.3. Abstraktierte Methoden
 - 5.6.2.4. Klassen-Methoden oder statische Methoden
 - 5.7. Sichtbarkeit
- 6. Klassen
 - 6.1. Klassen-Definitionen
 - 6.2. Abstrakte und Sealed Klassen
 - 6.3. Normale und statische Felder
 - 6.3.1. Normalisierte Felder / Variablen

- 6.3.2. Klassen-Felder / Variablen
- 6.4. Klassen - CTOR
- 6.5. Klassen - DTOR
- 6.6. Methoden
 - 6.6.1. Erklärung
 - 6.6.2. Aufrufen
 - 6.6.3. Virtuelle Methoden
 - 6.6.4. Klassen - Methoden
 - 6.6.5. Klassen CTOR und DTOR
 - 6.6.6. Statische Klassen - Methoden
 - 6.6.7. Nachrichten - Methoden
 - 6.6.8. Vererbung
- 6.7. Eigenschaften
 - 6.7.1. Definition
 - 6.7.2. Indezierte Eigenschaften
 - 6.7.3. Array basierte Eigenschaften
 - 6.7.4. Standard - Eigenschaften (public)
 - 6.7.5. Veröffentlichte - Eigenschaften (published)
 - 6.7.6. Speicherinformationen
 - 6.7.7. Eigenschaften überschreiben und neu deklarieren
- 6.8. Klassen - Eigenschaften
- 6.9 Verschachtelte Typen, Konstanten, und Variablen
- 7. Schnittstellen (Interface's)
 - 7.1. Definition
 - 7.2. Identifikation
 - 7.3. Implementierung
 - 7.4. Vererbung
 - 7.5. Delegation
 - 7.6. COM
 - 7.7. CORBA und andere Schnittstellen
 - 7.8. Referenzzählung
- Generics
 - 8.1. Einführung
 - 8.2. Get'ter Typ - Definition
 - 8.3. Typen - Spezialisierung
 - 8.4. Einschränkungen
 - 8.5. Kompatibilität zu Delphi
 - 8.5.1. Syntax - Elemente
 - 8.5.2. Einschränken für Record's
 - 8.5.3. Typen - Überladung(en)
 - 8.5.4. Überlegungen für Namensbereiche
 - 8.6. Typen-Kompatibilität
 - 8.7. Verwendung der eingebauten Funktionen
 - 8.8. Überlegungen zum Geltungsbereich
 - 8.9. Operator-Überladung und Generics
- 9. Erweiterte Record's
 - 9.1. Definition
 - 9.2. Erweiterte Record-Aufzähler
 - 9.3. Record-Operationen
- 10. Klassen, Record's, und Typen-Helfer
 - 10.1 Definition
 - 10.2 Einschränkungen bei Klassen Helfer
 - 10.3 Einschränkungen bei Record Helfer

- 10.4 Überlegungen zu einfachen Helper
- 10.5 Anmerkungen zu Umfang und Lebensdauer von Record-Helper
- 10.6 Vererbung
- 10.7 Verwendung
- 11. Objektorientierte Pascal - Klassen
 - 11.1. Einführung
 - 11.2. Klassendeklarationen
 - 11.3. Formele Deklaration
 - 11.4. Instanzen zuteilen und zuordnen
 - 11.5. Protokolldefinitionen
 - 11.6. Kategorien
 - 11.7. Namensumfang und Bezeichner
 - 11.8. Selektoren
 - 11.9. Der ID Type
 - 11.10. Aufzählungen in Objective-C Klassen
- 12. Ausdrücke
 - 12.1. Ausdrucks - Syntax
 - 12.2. Funktionsaufrufe
 - 12.3. Mengen - CTOR
 - 12.4. Typ-Casting von Werten
 - 12.5. Typ-Casting von Variablen
 - 12.6. Sonstige Typ-Casting's
 - 12.7. Der @ - Operator
 - 12.8. Operatoren
 - 12.8.1. Arithmetische Operatoren
 - 12.8.2. Logische Operatoren
 - 12.8.3. Boolesche Operatoren
 - 12.8.4. Zeichenketten Operatoren
 - 12.8.5. Operatoren bei dynamischen Array's
 - 12.8.6. Mengen - Operatoren
 - 12.8.7. Relationale Operatoren
 - 12.8.8. Klassen - Operatoren
- 13. Anweisungen
 - 13.1. Einfache Anweisungen
 - 13.1.1. Zuweisungen
 - 13.1.2. Prozeduren - PROCEDURE
 - 13.1.3. Sprungs - Anweisung GOTO
 - 13.2. Strukturierte Anweisungen
 - 13.2.1. Zusammengesetzte Anweisungen
 - 13.2.2. CASE
 - 13.2.3. IF ... THEN
 - 13.2.4. FOR ... TO / DOWNT0 ... DO
 - 13.2.5. FOR .. IN .. DO
 - 13.2.6. REPEAT ... UNTIL
 - 13.2.7. WHILE ... DO
 - 13.2.8. WITH
 - 13.2.9. Ausnahmen (EXCEPT)
 - 13.3. Assembler Anweisungen
- 14. Benutzung von Funktionen und Prozeduren
 - 14.1. FUNCTION Deklarationen
 - 14.2. PROCEDURE Deklarationen
 - 14.3. Funktion Rückgabewert mittels RESULT
 - 14.4. Parameter Listen

- 14.4.1. Werte
- 14.4.2. Variablen
- 14.4.3. Ausgabe Parameter
- 14.4.4. Konstante Parameter
- 14.4.5. Offene Array's
- 14.4.6. Array of Const
- 14.4.7. Untypisierte Parameter
- 14.4.8. Verwaltete Typen und Referenzzähler
- 14.5. Überladung von Funktionen
- 14.6. Mit FORWARD deklarierte Funktionen
- 14.7. Externe Funktionen
- 14.8. Assembler Funktionen
- 14.9. Modifikatoren
 - 14.9.1. alias
 - 14.9.2. cdecl
 - 14.9.3. cppdecl
 - 14.9.4. export
 - 14.9.5. hardfloat
 - 14.9.6. inline
 - 14.9.7. interrupt
 - 14.9.8. iocheck
 - 14.9.9. local
 - 14.9.10. MS_ABI_Default
 - 14.9.11. MS_ABI_CDecl
 - 14.9.12. MWPascal
 - 14.9.13. noreturn
 - 14.9.14. nostackframe
 - 14.9.15. overload
 - 14.9.16. pascal
 - 14.9.17. public
 - 14.9.18. register
 - 14.9.19. safecall
 - 14.9.20. saveregisters
 - 14.9.21. softfloat
 - 14.9.22. stdcall
 - 14.9.23. SYSV_ABI_Default
 - 14.9.24. SYSV_ABI_CDecl
 - 14.9.25. VectorCall
 - 14.9.26. varargs
 - 14.9.27. winapi
- 14.10. Nicht unterstützte Modifikatoren
- 15. Operatoren Überladung
 - 15.1. Einleitung
 - 15.2. Operatoren - Deklarationen
 - 15.3. Operator - Zuweisung
 - 15.4. Arithmetische Operatoren
 - 15.5. Vergleichende Operatoren
 - 15.6. In Operator
 - 15.7. Logik Operatoren
 - 15.8. Auf- und Ab-Zählungs Operatoren
 - 15.9. Aufzählungs - Operator
- 16. Programme, Module und Blöcke
 - 16.1. Programme

- 16.2. Module (Unit's)
- 16.3. Namensräume
- 16.4. Abhängigkeiten von Modulen
- 16.5. Blöcke
- 16.6. Anwendungsbereiche (Scope)
 - 16.6.1. Blöcke
 - 16.6.2. Record's
 - 16.6.3. Klassen
 - 16.6.4. Unit's
- 16.7. Bibliotheken
- 17. Ausnahmen
 - 17.1. Die RAISE Anweisung
 - 17.2. Ausnahme-Behandlung und Verschachtelung
- 18. Assembler
 - 18.1. Anweisungen
 - 18.2. Prozeduren und Funktionen

Liste der Tabellen

Über dieses Handbuch

Dieses Dokument dient als Referenz für die Sprache Pascal, wie sie durch den Free Pascal Compiler implementiert ist. Es beschreibt alle Pascal-Konstrukte, die von Free Pascal unterstützt werden, und listet alle unterstützten Datentypen auf. Es vermittelt jedoch keine detaillierten Erklärungen der Pascal-Sprache: Es ist kein Anleitungsheft.

Das Ziel ist es, zu zeigen, welche Pascal-Konstrukte unterstützt werden und wo sich die Free Pascal-Implementierung von den Turbo Pascal- oder Delphi-Implementierungen unterscheidet.

Die Turbo Pascal und Delphi Pascal Compiler führten verschiedene Features in die Pascal Sprache ein. Der Free Pascal Compiler imitiert diese Compiler im entsprechenden Modus des Compilers: bestimmte Funktionen sind nur verfügbar, wenn der Compiler in den entsprechenden Modus versetzt wird.

Wenn ein bestimmtes Feature benötigt wird, ist die Verwendung des **-M** Kommandozeilenschalters oder der Direktive **{ \$MODE }** im Quelltext erforderlich. Weitere Informationen über die verschiedenen Einstellungen finden Sie im Benutzerhandbuch und im Programmiererhandbuch.

Bezeichnungen

Bezeichnungen

In diesem Dokument werden Funktionen, Typen und Variablen mit Schreibmaschinenschrift gekennzeichnet.

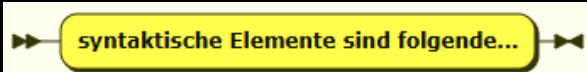
Auf Dateien wird mit einer Sans-Schrift Bezug genommen: Dateiname.

Syntax Diagramme

Diagramme zur Syntax

Alle Elemente der Sprache Pascal werden in Syntaxdiagramme erklärt. Syntaxdiagramme sind wie Flussdiagramme. Das Lesen eines Syntaxdiagramms bedeutet, von der linken Seite zur rechten Seite zu gelangen, indem man den Pfeilen folgt. Wenn die rechte Seite eines Syntaxdiagramms erreicht ist und mit einem einzelnen Pfeil endet, bedeutet, dass das Syntaxdiagramm in der nächsten Zeile fortgesetzt wird. Endet die Zeile an zwei aufeinander zeigenden Pfeilen dann ist das Diagramm beendet.

Syntaktische Elemente werden wie folgt geschrieben:



Schlüsselwörter, die genau wie im Diagramm eingegeben werden müssen:



Wenn etwas wiederholt werden kann, ist es mit einem Pfeil gekennzeichnet:

Schlüsselwort (keyword):	EBNF:
	<pre>keyword ::= "Schlüsselwort" keyword</pre>

Wenn es verschiedene Möglichkeiten gibt, werden diese in Zeilen aufgelistet:

verschiedene Möglichkeiten 1:	EBNF:
	<pre>Möglichkeiten ::= "Möglichkeit A" "Möglichkeit B" "Möglichkeit C" "..."</pre>

Beachten Sie, dass eine der Möglichkeiten leer sein kann:

verschiedene Möglichkeiten 2:	EBNF:
	<pre>Möglichkeiten ::= "Möglichkeit A" "Möglichkeit B" "Möglichkeit C" "..."</pre>

Das bedeutet, dass sowohl die erste als auch die zweite Möglichkeit optional sind. Natürlich können alle diese Elemente kombiniert und verschachtelt werden.

Über die Sprache Pascal

Die Sprache Pascal wurde ursprünglich um 1970 von Niklaus Wirth entwickelt. Seitdem hat sie sich deutlich weiterentwickelt (mit vielen Beiträgen der verschiedenen Compiler-Entwickler - insbesondere: BORLAND). Die Grundelemente wurden über die Jahre hinweg beibehalten:

- einfache Satzstruktur, recht ausführlich, aber leicht zu lesen. Ideal für den Unterricht.

- starke Typisierung.
- prozedural.
- Groß- und Kleinschreibung wird nicht berücksichtigt.
- erlaubt verschachtelte Prozeduren
- einfache integrierte Ein- und Ausgaberroutinen.

1. Pascal Zeichen und Symbole

Zeichen sind die grundlegenden lexikalischen Bausteine des Quellcodes: Sie sind die „Wörter“ der Sprache: Zeichen werden gemäß den Regeln der Programmiersprache zu Token kombiniert. Es gibt fünf Klassen von Zeichen:

- **reservierte Wörter**
Dies sind (Schlüssel)-Wörter die eine feste Bedeutung innerhalb der Sprache Pascal darstellen. Sie können nicht geändert oder neu definiert werden.
- **Bezeichner**
Dies sind Namen von Symbolen die der Programmierer definieren kann.
Sie können geändert und wiederverwendet werden.
Sie unterliegen dem Geltungsbereichsregeln der Sprache Pascal
- **Operatoren**
Dies sind in der Regel Symbole für mathematische oder andere Operationen.
Im allgemeinen betrifft dies die Symbole *, +, - und so weiter.
- **Trennzeichen**
Dies sind im allgemeine Leerzeichen
- **Konstanten**
Numerische oder Zeichenkonstanten werden verwendet, um tatsächliche Werte im Quellcode zu bezeichnen, wie beispielsweise 1 (ganzzahlige Konstante) oder 2,3 (Gleitkommakonstante) oder 'Zeichenkette'

2. Konstanten

3. Typen

4. Variablen

5. Objekte

6. Klassen

[7. Schnittstellen \(Interface's\)](#)[8. Generics](#)[9. Erweiterte Record's](#)[10. Klassen, Records, und Typen-Helfer](#)[11. Objektorientierte Pascal - Klassen](#)[12. Ausdrücke](#)[13. Anweisungen](#)[14. Benutzung von Funktionen und Prozeduren](#)[15. Operatoren - Überladung](#)[16. Programme, Module, und Blöcke](#)[17. Ausnahmen](#)[18. Assembler](#)[18.1. Anweisungen](#)[18.2. Prozedure und Funktionen](#)